

TaxaTools

Shared taxonomic-name and LLM-provider utilities for the TaxaID ecosystem. Every other TaxaID package imports TaxaTools; it has no dependency on any of them, which makes it the ecosystem’s foundation package. It can also be used standalone for cleaning and verifying taxon name lists, resolving synonyms across backbones, or calling an LLM from a plain R script with no other TaxaID package installed.

TaxaTools solves three largely independent problems that every downstream package needs solved consistently:

- **Taxonomic names arrive messy and inconsistent.** Field data sheets, GenBank records, and citizen-science exports spell the same species differently, encode binomials as `Genus_epithet`, attach author citations, or use a synonym one backbone has since retired. For these reasons, `verify_taxon_names()`, `clean_taxon_names()`, and `change_backbone()`’s companion functions reconcile names against a chosen taxonomic backbone (GBIF; NCBI, the National Center for Biotechnology Information, U.S. National Library of Medicine, National Institutes of Health, Bethesda, Maryland; WoRMS, the World Register of Marine Species, Flanders Marine Institute (VLIZ), Ostend, Belgium; Catalogue of Life, hosted by Naturalis Biodiversity Center, Leiden, Netherlands; or ITIS, the Integrated Taxonomic Information System). This makes it possible to join species lists from different source (such as when combining priors and likelihoods to generate posteriors). Original names can be kept to maintain connections with the source data.
- **Every LLM-calling function in the ecosystem needs one interface, not four.** TaxaHabitat’s habitat assignment, TaxaAssign’s LLM-shortcut pipeline, and TaxaFlag’s expert review all submit custom prompts to a language model and none of them should have to know which provider is configured. `call_api()` dispatches to Anthropic Claude, Google Gemini, OpenAI, Azure OpenAI, or a local Ollama model behind one signature, auto-detected from whichever API key is present in `~/Renviro`. Because these LLMs are constantly evolving, this aspect of the package is designed to be neutral with respect to model names as much as possible, but with the advent of new LLM models, the user may need to tailor these functions to meet new opportunities.
- **Several functions are reused and repackaged in different downstream packages.** Barcode/primer registries, common-name lookup, on-disk cache management, report-text drafting, and a shared interactive spatial gadget exist because more than one downstream package needed the same small piece of infrastructure and duplicating it would have let the copies drift.

Function Inventory

Taxonomy verification and cleaning

Function	Purpose
<code>verify_taxon_names()</code>	Verify names against a taxonomic backbone via the Global Names Verifier API (batched). Returns <code>matched_name</code> , <code>matched_rank</code> , <code>is_synonym</code> , <code>classification_path</code> , <code>score</code> . Prefers a backbone’s currently-accepted name over a retired synonym.
<code>clean_taxon_names()</code>	Normalise, deduplicate, and filter a character vector of names – drops NA, non-capitalized, abbreviated, and bracket-artefact entries; converts underscore-encoded binomials (<code>Genus_epithet</code> , common in Jonah Ventures/SILVA output) to space-separated form.
<code>create_taxon_names()</code>	Add <code>taxon_name/taxon_name_rank</code> columns to a data frame from separate rank columns (most-specific non-NA rank wins).
<code>change_backbone()</code>	Reshape <code>verify_taxon_names()</code> output into wide rank columns, renaming source/translated name columns.

Function	Purpose
<code>fill_higher_ranks()</code>	Given species binomials, look up genus and family via a priority chain: local lookup tables, then a primary backbone, then a fallback backbone.
<code>escalate_taxonomic_rank()</code>	Broaden a taxon to the next coarser rank (genus -> family -> order) when nothing exists at the current rank – e.g. a singleton with no reference sequences at species level.
<code>parse_classification_path()</code>	Extract one rank's value from the pipe-delimited classification columns <code>verify_taxon_names()</code> returns.
<code>find_taxonomy_conflicts()</code>	Detect higher-rank inconsistencies in a taxonomy table (e.g. one genus mapped to two different families).
<code>is_plausible_binomial()</code>	Filter out "sp.", "cf.", "aff.", uncultured/environmental labels, and other non-binomial names.
<code>to_faure()</code>	Export a TaxaID match/likelihood/posterior table to FAIR checklist column conventions.

LLM provider interface

Function	Purpose
<code>call_api()</code>	Generic dispatcher: one prompt string (plus optional images) to whichever provider is configured. Handles Anthropic, Gemini, and any OpenAI-compatible endpoint (OpenAI, Ollama). Attaches token-usage and provider/model attributes to the response.
<code>call_anthropic_api()</code> , <code>call_gemini_api()</code> , <code>call_openai_api()</code> , <code>call_azure_openai_api()</code> , <code>call_ollama_api()</code>	Thin provider-specific wrappers around <code>call_api()</code> , kept for direct use.
<code>prompt_api()</code>	Multi-chunk prompt dispatcher; default <code>llm_fn</code> read from <code>getOption("TaxaID.llm_fn")</code> .
<code>prompt_manual()</code> / <code>read_llm_response()</code>	Write a prompt to a file for manual submission via a web interface, then read the saved response back in – for workflows without a paid API key.
<code>token_usage()</code> / <code>reset_token_usage()</code>	Per-call, per-function, per-provider, or per-session token accounting; optional cost estimate.
<code>% %</code>	Null-coalescing operator, exported for every downstream package to import.

On `library(TaxaTools)`, `.onAttach()` scans `~/Renvi` for API keys (priority: Anthropic > Gemini > OpenAI) and sets `options(TaxaID.llm_fn = <detected provider>)` automatically, so no downstream function needs its own provider-detection logic.

Rank and barcode utilities

Function	Purpose
<code>standard_ranks</code> / <code>extended_ranks</code>	Canonical rank vectors (kingdom through species; extended adds subspecies/variety/form).
<code>detect_ranks()</code>	Auto-detect which rank columns exist in a data frame.

Function	Purpose
<code>barcode_length_defaults</code>	Named list of barcode markers (12S/16S/COI/cytb/ITS/rbcL/matK/trnL/...) to expected amplicon-length ranges.
<code>resolve_barcode_lengths()</code>	Resolve a min/max bp window from one or more <code>barcode_term</code> values.
<code>barcode_primer_defaults</code>	Named list of <i>specific</i> primer variants (e.g. "mifish-u", "coi-leray") to <code>list(fwd, rev, amplicon_range)</code> , each verified against literature and a real GenBank sequence before being added.
<code>resolve_barcode_primers()</code>	Resolve <code>fwd/rev/amplicon_range</code> for a specific variant; errors rather than guessing on an ambiguous bare term (e.g. "mifish" alone) or an unregistered marker.
<code>resolve_barcode_marker()</code>	Resolve a registered primer-variant term (e.g. "COI-Folmer") to the marker it amplifies, for building a sequence-database query term – a variant name is not itself searchable in GenBank.

Common names and LLM text generation

Function	Purpose
<code>common_to_scientific()</code>	Convert common names to scientific names via LLM, with optional backbone verification.
<code>scientific_to_common()</code>	Convert scientific names to English common names via a taxonomic backbone (GBIF or ITIS) with LLM fallback; <code>location</code> biases toward regionally appropriate names. <code>cache_dir</code> keeps one small <code>.rds</code> per name so a re-run asks nothing twice; <code>verbose</code> prints a summary and one line per LLM batch.
<code>taxatools_clear_cache()</code>	Report and prune a <code>scientific_to_common(cache_dir =)</code> directory (built on <code>list_cache_files()/report_and_clear_cache()</code>).
<code>build_report_context()</code>	Domain-agnostic context object carrying verified facts for grounding LLM-drafted text.
<code>draft_methods_text()</code> / <code>draft_results_text()</code>	Read R code or R objects and draft a Methods or Results section via LLM.
<code>census_genus_species()</code>	Enumerate described species per genus (or higher rank) via the GBIF backbone; flags whether a reference set is complete, missing only its rarest member, or genuinely incomplete.

Cache management and interactive gadget

Function	Purpose
<code>list_cache_files()</code> / <code>report_and_clear_cache()</code>	Shared engine behind a downstream package's own <code><pkg>_clear_cache()</code> helper (e.g. <code>TaxaFetch::taxafetch_clear_cache()</code> , <code>TaxaLikely::taxalikely_clear_cache()</code>) – scans a cache directory, reports age/size, and deletes or dry-run-reports what's stale.
<code>define_search_polygon()</code>	Interactive Shiny/leaflet gadget: drag corner markers to define a custom search polygon, returned as a WKT string. Shared by <code>TaxaFetch</code> 's search-area fetches and <code>TaxaMatch::group_observations_by_bbox()</code> 's spatial grouping.

Installation

TaxaTools has no dependency on any other TaxaID package, so it should be installed first:

```
devtools::install("path/to/TaxaTools")
```

API Setup

Most of TaxaTools works with no key at all (`clean_taxon_names()`, `create_taxon_names()`, backbone verification via the public Global Names Verifier API, barcode/rank utilities). Two categories of function need one:

- **LLM functions** (`call_api()` and anything built on it – `common_to_scientific()`, `draft_methods_text()`, and every LLM-calling function across the ecosystem) need a key from at least one provider: Anthropic Claude (`ANTHROPIC_API_KEY`, the default), Google Gemini (`GEMINI_API_KEY`, has a free tier), OpenAI (`OPENAI_API_KEY`), Azure OpenAI (`AZURE_OPENAI_API_KEY`, DOI employees only), or none at all if you run a local Ollama model.
- **NCBI-backed functions** (`verify_taxon_names(backbone_id = 4)`, and downstream in Taxa-Likely/TaxaMatch) work without a key but raise NCBI's rate limit from 3 to 10 requests/second with one (`ENTREZ_KEY`).

Set keys in `~/.Renviron` (one per line, no quotes):

```
usethis::edit_r_environ()
```

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

```
ENTREZ_KEY=your_ncbi_api_key_here
```

Restart R after editing – environment variables are only read at startup. On `library(TaxaTools)`, `.onAttach()` scans for whichever LLM key(s) it finds and prints which provider was auto-detected; verify with `getOption("TaxaID.llm_fn")`. See the API Setup vignette for the complete key list used across the whole TaxaID ecosystem (GBIF, OpenAlex, etc.), where to get each one, and troubleshooting for common errors (401/429, truncated responses, key not detected).

Quick Start

Clean and verify a messy name list

```
library(TaxaTools)

raw_names <- c("Gadus_morhua", "sebastes sp.", "Paracalanus parvus (Claus, 1863)")

cleaned <- clean_taxon_names(raw_names)
#> "Gadus morhua" "Paracalanus parvus" (the "sp." entry is dropped)

verified <- verify_taxon_names(cleaned, backbone_id = 11) # GBIF
# verified$matched_name / matched_rank / is_synonym / classification_path
```

Verify the same name against different taxonomic backbones

`backbone_id` selects which authority resolves a name. Backbones don't always agree – a name can be a live species in one and a retired synonym in another (`is_synonym/matched_rank` will differ), which is exactly why downstream packages let the caller choose rather than hardcoding one:

backbone_id	Backbone
1	Catalogue of Life
3	ITIS (Integrated Taxonomic Information System)
4	NCBI
9	WoRMS (World Register of Marine Species)
11	GBIF

```
# Same query, five backbones -- compare matched_name/is_synonym across them
backbones <- c(col = 1, itis = 3, ncbi = 4, worms = 9, gbif = 11)

lapply(backbones, function(id) {
  verify_taxon_names("Urobatis halleri", backbone_id = id)[
    , c("matched_name", "matched_rank", "is_synonym")
  ]
})
```

A mismatch here isn't a bug in `verify_taxon_names()` – it's telling you the backbones themselves disagree, which matters when joining data that was verified against different ones (e.g. sequence references verified against NCBI, occurrence records verified against GBIF).

Call an LLM without knowing which provider is configured

```
# .onAttach() already set options(TaxaID.llm_fn) from your ~/.Renvirom
response <- call_api("Summarize the habitat preferences of Gadus morhua in one sentence.")
```

Resolve a barcode marker's expected amplicon length and primers

```
resolve_barcode_lengths("MiFish")           # list(min = 163L, max = 185L)
resolve_barcode_primers("MiFish-U")         # list(fwd = ..., rev = ..., amplicon_range = ...)
resolve_barcode_marker("COI-Folmer")        # "COI" -- the term a sequence database actually indexes
```

Convert between common and scientific names

```
common_to_scientific(c("Pacific cod", "sablefish"), taxonomic_group = "fish")
scientific_to_common("Gadus macrocephalus", backbone_id = 11)
```

Typical Workflow

```

rename_cols()           # align column names to DarwinCore
|> create_taxon_names() # derive best taxon name per row
|> clean_taxon_names()  # deduplicate & clean for API submission
|> verify_taxon_names() # check against a taxonomic backbone
|> change_backbone()    # reshape into wide taxonomy columns

```

Vignettes

- API Setup – configuring an LLM provider key and confirming auto-detection
- Name Cleaning – the full clean/verify/backbone-convert pipeline

Part of TaxaID

TaxaTools is the foundation of the TaxaID ecosystem – every other package imports it, and it imports none of them:

```

TaxaTools -> TaxaFetch -> TaxaHabitat -> TaxaExpect -> TaxaAssign -> TaxaFlag
TaxaTools -> TaxaMatch -> TaxaLikely -> TaxaAssign -> TaxaFlag

```

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$; R Core Team 2025)
- An Application Programming Interface (API) key for an LLM provider (Anthropic Claude, Google Gemini, OpenAI, or local Ollama with no key) is needed for `call_api()` and every function that calls it (`common_to_scientific()`, `scientific_to_common()`'s LLM fallback, `draft_methods_text()`, `draft_results_text()`)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic PBC, San Francisco, California).

References

R Core Team (2025). R: A Language and Environment for Statistical Computing. V.4.5.2. R Foundation for Statistical Computing, Vienna, Austria. <https://www.r-project.org>